

THÈSE

Pour obtenir le grade de

DOCTEUR DE L'UNIVERSITÉ GRENOBLE ALPES



École doctorale : EEATS - Electronique, Electrotechnique, Automatique, Traitement du Signal

Spécialité : Nano électronique et Nano technologies

Unité de recherche : CEA/List/DSCIN

Conception d'un Système Mémoire pour Traitement de Données Creuses

Design of a Memory System for Sparse Data Processing

Présentée par :

Valentin ISAAC--CHASSANDE

Direction de thèse :

Frédéric ROUSSEAU

PROFESSEUR DES UNIVERSITES, UNIVERSITE GRENOBLE ALPES

Directeur de thèse

Adrian EVANS

CEA

Co-encadrant de thèse

Rapporteurs :

Thèse soutenue publiquement le , devant le jury composé de :



Valentin ISAAC--CHASSANDE

Université Grenoble Alpes
CEA Grenoble (DRT/LIST/DSCIN/LSTA)
TIMA Laboratory (SLS)

ORCID : 0009-0007-9842-5777

valentin.isaacchassande@univ-grenoble-alpes.fr
valentin.isaacchassande@cea.fr

THESIS

DESIGN OF A MEMORY SUB-SYSTEM FOR SPARSE DATA PROCESSING

February 27, 2025

Thesis Adviser: Frédéric ROUSSEAU
Thesis Co-Supervisors: Adrian EVANS
Yves DURAND

CSI Members: Yves ROBERT (HDR)
Christophe PICARD

EEATS Doctoral College, NENT Speciality
2022 – 2025 Academic Years
From 10.25.2022 to 10.24.2025

Acknowledgements

TODO

DRAFT

TODO *Quote here. Quote here. Quote here. Quote here. Quote here. Quote here.*
Quote here. Quote here. Quote here. Quote here.

Author, "Title".

Résumé

TODO

DRAFT

Abstract

TODO

DRAFT

List of Publications

This thesis is based on the following publications:

- I Valentin Isaac--Chassande, Adrian Evans, Yves Durand, and Frédéric Rousseau. 2024. Dedicated Hardware Accelerators for Processing of Sparse Matrices and Vectors: A Survey. *ACM Trans. Archit. Code Optim.* 21, 2, Article 27 (Feb. 2024), 26 pages. Reference [5]
- II Valentin Isaac--Chassande, Adrian Evans, Yves Durand, and Frédéric Rousseau. 2024. SpDCache: Region-Based Reduction Cache for Outer-Product Sparse Matrix Kernels. In *2024 IEEE 35th International Conference on Application-specific Systems, Architectures and Processors (ASAP)*. IEEE Computer Society, Los Alamitos, CA, USA, 3–7. Reference [6]
- III **TODO**

The papers will be referred to in the thesis using their Roman numerals.

Summary

Acknowledgements	1
Epigraph	2
Résumé	3
Abstract	4
List of Publications	5
Summary	6
Notations	7
1 Plan (Brouillon)	8
2 SoA	10
Conclusion	11
1 TODO	11
Appendices	12
A TODO	12
List of Figures	13
List of Tables	14
List of Algorithms	15
List of Source Code	16
Bibliography	17
Contents	19

Notations

$$\forall (a, b) \in \mathbb{Z}^2, \llbracket a, b \rrbracket = [a, b] \cap \mathbb{Z}$$

TODO

Chapter 1

Plan (Brouillon)

- SoA
 - irregular accesses -> def, issues (in memory but not only?), domains/applications (including sparse matrix processing)
 - Focus on sparse matrices, context: it is widely used, in many domains, sparse, large, structures
 - Sparse matrices in a kernel: show where the accesses are regular vs irregular (infos on basic sparse formats, basic kernels, basic algos...)
 - Hardware challenges clearly defined (gather, scatter...)
 - Present solutions in SW for CPU/GPU, such as special sparse formats, possibility to play on algo (gust analysis), tiling methods adapted for sparse data (specific sparse format + specific algos)
 - Present solutions in HW (dedicated accelerators) -> classification along common characteristics
 - Presentation of the major accelerators
 - Problem of comparison / comparison of standalone / other analysis (issue of matrix structure)
 - Takeaways -> paths for designing an accelerator (end of SoA paper)
 - We choose a path -> show which one (coarse grain, at the level of the SoA, comparable to SortCache, PHI...)
- Theory
 - Describe the design space of the path we choose (with remainders if necessary) -> cache / reductions / OP SpMV / ...
 - Emphasis the comparison between concerned SoA accelerators -> show what is already done and useful, show what is missing
 - Explain the coarse grain concept of our solution compared to the previous ones (regions)
 - Show the interest of separate cache regions for A/b and c (formal) -> show that a GC is obviously not efficient when random accesses / confirm why SortCache, PHI, ...
 - Show that A and b need a 1 set 4 ways region -> it is not a limiting factor
 - We use a GRC (no modification) for A and b -> quick formalisation to get nb loads and writes, hit, miss, evict
 - Show the interest of a reduction cache region for c -> REDs / reduce the number of mem accesses / link to SoA
 - Show that it is not enough to handle randomness of REDs -> we propose

- additional regions
- "Randomness" -> tied to the matrix structure -> coarse grain (ex banded) / fine grain locality VS. isolated elements
- To show the interest of having regions in RedC, model of RedC behaviour, to use with different configurations (give some already, explain what we expect to see, and what additional info it will give us (cache parameters analysis))
- RedC Modelling / Validation compare to Python model / Advantage of having the proba model
- Show RedC 1reg vs RedC 2reg on banded example
- Show best cache dimensions for the dense part (band)
- Show that sparse region (out of band) could use an other region for fine grain density (case of sub bands) -> dimensions
- Show that sparse region could use adapted HW because of the isolated elements -> dimensions
- Show how it would work with a "full random" matrix -> how does it affects the dimensions
- Explain the concept of our solution with details on the regions, the dimensions, the case study (banded matrix)
- Hardware solution
 - Present our hardware solution, based on the previous concept (remind) / several architecture
 - SpDCCache, architecture presentation with two regions, details on REDs transactions, IBRC vs OBRT
 - Architecture presentation with three regions (add OBRC, CB...)
 - Architecture of eviction anticipation
 - Present multicore version -> scale
 - Present additional optimisation (region config, learn from first iteration)
- Experimentations
 - Present our implementation -> only SpDCCache, based on HPDcache in open source
 - Experimentations -> matrices chosen, generated / metrics / ...
 - Results / comparison with theory / analysis / conclusion

Chapter 2

SoA

...

DRAFT

Conclusion

1 TODO

TODO

DRAFT

Appendices

A TODO

TODO if necessary

DRAFT

List of Figures

DRAFT

List of Tables

DRAFT

List of Algorithms

DRAFT

List of Source Code

DRAFT

Bibliography

- [1] Richard Barrett, Michael Berry, Tony F. Chan, James Demmel, June Donato, Jack Dongarra, Victor Eijkhout, Roldan Pozo, Charles Romine, and Henk van der Vorst. 1994. *Templates for the Solution of Linear Systems: Building Blocks for Iterative Methods*. Society for Industrial and Applied Mathematics. <https://doi.org/10.1137/1.9781611971538>
- [2] Scott Beamer, Krste Asanović, and David Patterson. 2017. The GAP Benchmark Suite. arXiv:1508.03619 [cs.DC] <http://gap.cs.berkeley.edu/benchmark.html>
- [3] Achi Brandt and Dorit Ron. 2003. *Multigrid Solvers and Multilevel Optimization Strategies*. Springer US, Boston, MA, 1–69. https://doi.org/10.1007/978-1-4757-3748-6_1
- [4] Mehmet Deveci, Simon David Hammond, Michael M. Wolf, and Sivasankaran Rajamanickam. 2018. Sparse Matrix-Matrix Multiplication on Multilevel Memory Architectures: Algorithms and Experiments. (4 2018). <https://doi.org/10.2172/1435688>
- [5] Valentin Isaac--Chassande, Adrian Evans, Yves Durand, and Frédéric Rousseau. 2024. Dedicated Hardware Accelerators for Processing of Sparse Matrices and Vectors: A Survey. *ACM Trans. Archit. Code Optim.* 21, 2, Article 27 (Feb. 2024), 26 pages. <https://doi.org/10.1145/3640542>
- [6] Valentin Isaac--Chassande, Adrian Evans, Yves Durand, and Frédéric Rousseau. 2024. SpDCache: Region-Based Reduction Cache for Outer-Product Sparse Matrix Kernels. In *2024 IEEE 35th International Conference on Application-specific Systems, Architectures and Processors (ASAP)*. IEEE Computer Society, Los Alamitos, CA, USA, 3–7. <https://doi.org/10.1109/ASAP61560.2024.00012>
- [7] JESD235A 2016. *JEDEC Updates Groundbreaking High Bandwidth Memory (HBM) Standard*. Technical Report. JEDEC. <https://www.jedec.org/news/pressreleases/jedec-updates-groundbreaking-high-bandwidth-memory-hbm-standard>
- [8] Shaden Smith, Jee W. Choi, Jiajia Li, Richard Vuduc, Jongsoo Park, Xing Liu, and George Karypis. 2017. *FROSTT: The Formidable Repository of Open Sparse Tensors and Tools*. <http://frostdt.io/>

- [9] Sriseshan Srikanth, Thomas M. Conte, Erik P. DeBenedictis, and Jeanine Cook. 2017. The superstrider architecture: Integrating logic and memory towards non-von Neumann computing. *2017 IEEE International Conference on Rebooting Computing, ICRC 2017 - Proceedings* 2017-January (2017), 1 – 8. <https://doi.org/10.1109/ICRC.2017.8123669>
- [10] Guowei Zhang, Nithya Attaluri, Joel S. Emer, and Daniel Sanchez. 2021. Gamma: Leveraging Gustavson’s Algorithm to Accelerate Sparse Matrix Multiplication. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (Virtual, USA) (ASPLOS ’21)*. Association for Computing Machinery, New York, NY, USA, 687–701. <https://doi.org/10.1145/3445814.3446702>

Contents

Acknowledgements	1
Epigraph	2
Résumé	3
Abstract	4
List of Publications	5
Summary	6
Notations	7
1 Plan (Brouillon)	8
2 SoA	10
Conclusion	11
1 TODO	11
Appendices	12
A TODO	12
List of Figures	13
List of Tables	14
List of Algorithms	15
List of Source Code	16
Bibliography	17
Contents	19